MINI PROJECT

FRUIT FRESHNESS

TEAM: BRINDHA B

DIVYADARSHINI S

DHARSAN ABINAV R K

Aim

To develop a deep learning-based image classification system to identify whether a fruit is **Fresh or Rotten** using Convolutional Neural Network (CNN) and MobileNetV2 models.

Project Idea

The project focuses on detecting fruit freshness from an input image.

The system takes an image of a fruit and classifies it into two categories:

- **Fresh**
- **Rotten**

Two deep learning models are used:

1. **CNN** – Baseline model developed from scratch.
2. **MobileNetV2** – Transfer learning model using a pre-trained ImageNet network.

The performance of both models is compared using **test accuracy**, and MobileNetV2 is used to predict the freshness of a new uploaded fruit image.

Dataset

A small public fruit freshness image dataset is used.

Classes:

- Fresh
- Rotten

The dataset is divided into:

- Training data – used for model training
- Validation data – used to monitor training performance
- Testing data – used for final evaluation

Images are resized to **224 × 224 pixels** before being given to the models.

Model Training

Model 1 – CNN

The CNN model contains:

- Image resizing and normalization
- Data augmentation
- Convolution layers
- Max pooling layers
- Fully connected layer
- Dropout layer
- Sigmoid output layer

The CNN is trained using the **Adam optimizer** and **Binary Cross-Entropy loss**.

Model 2 – MobileNetV2

MobileNetV2 is a pre-trained CNN architecture.

The ImageNet-trained base model is used as a feature extractor, and a new classification layer is added for **Fresh/Rotten** prediction.

MobileNetV2 is trained using:

- Adam optimizer
- Binary Cross-Entropy loss
- 10 epochs
- Frozen pre-trained layers

Model Training Code

The complete Colab code trains both models, compares their accuracy, generates a classification report and confusion matrix, and allows the user to upload a fruit image for prediction.

```
history_cnn = cnn.fit(

train_data,

validation_data=valid_data,

epochs=15
```

```
)

cnn_loss, cnn_accuracy = cnn.evaluate(
   test_data,
verbose=0
)

history_mobilenet = mobilenet.fit(
train_data,     validation_data=valid_data,
epochs=10
)

mobile_loss, mobile_accuracy = mobilenet.evaluate(
   test_data,
verbose=0
)

print("CNN Test Accuracy:",
round(cnn_accuracy * 100, 2), "%")

print("MobileNetV2 Test Accuracy:",
round(mobile_accuracy * 100, 2), "%")
For a new image:
uploaded = files.upload()

filename = list(uploaded.keys())[0]
```

```
img = load_img(
    filename,
    target_size=(224, 224)
)

img_array = img_to_array(img)

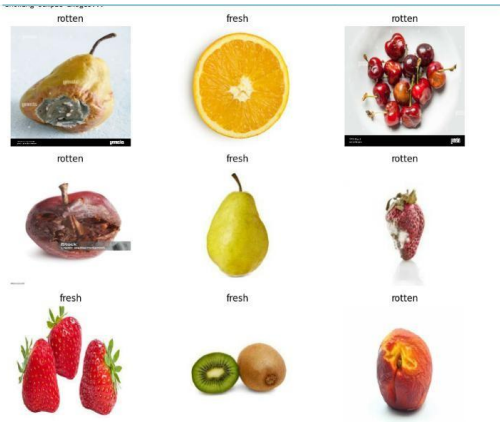
img_array = np.expand_dims(
img_array,     axis=0
)

prediction = mobilenet.predict(
img_array,     verbose=0
)[0][0]

if prediction > 0.5:     result =
"ROTTEN"     confidence =
prediction * 100 else:
    result = "FRESH"     confidence =
(1 - prediction) * 100

print("Prediction:", result)
print("Confidence:", round(confidence, 2), "%")
```

OUTPUT:




```
==============================
       SIMPLE CNN MODEL
==============================
Model: "sequential_4"
```

Layer (type)	Output Shape	Param #
sequential_3 (Sequential)	(None, 224, 224, 3)	0
rescaling_2 (Rescaling)	(None, 224, 224, 3)	0
conv2d_3 (Conv2D)	(None, 222, 222, 32)	896
max_pooling2d_3 (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_4 (Conv2D)	(None, 109, 109, 64)	18,496
max_pooling2d_4 (MaxPooling2D)	(None, 54, 54, 64)	0
conv2d_5 (Conv2D)	(None, 52, 52, 128)	73,856
max_pooling2d_5 (MaxPooling2D)	(None, 26, 26, 128)	0
flatten_1 (Flatten)	(None, 86528)	0
dense_3 (Dense)	(None, 128)	11,075,712
dropout_2 (Dropout)	(None, 128)	0
dense_4 (Dense)	(None, 1)	129

```
Total params: 11,169,089 (42.61 MB)
Trainable params: 11,169,089 (42.61 MB)
Non-trainable params: 0 (0.00 B)
```

```
============================
   MOBILENETV2 MODEL
============================
Model: "sequential_5"
```

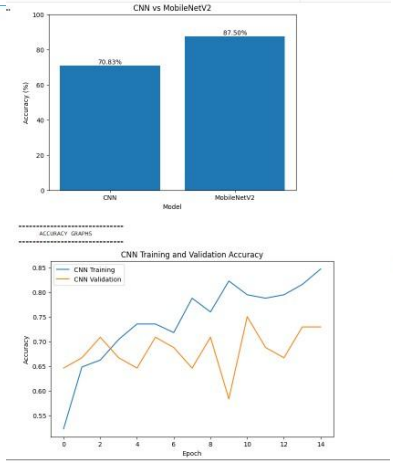
Layer (type)	Output Shape	Param #
rescaling_3 (Rescaling)	(None, 224, 224, 3)	0
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 1280)	0
dropout_3 (Dropout)	(None, 1280)	0
dense_5 (Dense)	(None, 1)	1,281

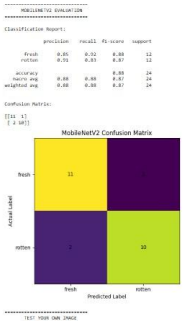
```
Total params: 2,259,265 (8.62 MB)
Trainable params: 1,281 (5.00 KB)
Non-trainable params: 2,257,984 (8.61 MB)

Training MobileNetV2...
Epoch 1/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 15s 454ms/step - accuracy: 0.6202 - loss: 0.6673 - val_accuracy: 0.6875 - val_loss: 0.5373
Epoch 2/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.7561 - loss: 0.4976 - val_accuracy: 0.8125 - val_loss: 0.4681
Epoch 3/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.8467 - loss: 0.3765 - val_accuracy: 0.8750 - val_loss: 0.3857
Epoch 4/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.8850 - loss: 0.3103 - val_accuracy: 0.8750 - val_loss: 0.3554
Epoch 5/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.9129 - loss: 0.2715 - val_accuracy: 0.8750 - val_loss: 0.3347
Epoch 6/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - accuracy: 0.9338 - loss: 0.2429 - val_accuracy: 0.8750 - val_loss: 0.3293
Epoch 7/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 2s 71ms/step - accuracy: 0.9443 - loss: 0.2066 - val_accuracy: 0.8750 - val_loss: 0.3277
Epoch 8/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.9617 - loss: 0.1824 - val_accuracy: 0.8750 - val_loss: 0.3148
Epoch 9/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.9582 - loss: 0.1678 - val_accuracy: 0.8750 - val_loss: 0.3051
Epoch 10/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.9582 - loss: 0.1649 - val_accuracy: 0.8750 - val_loss: 0.3030

MobileNetV2 Test Accuracy: 87.5 %

==========================
    MODEL COMPARISON
==========================
CNN Accuracy       : 70.83 %
MobileNetV2 Accuracy: 87.5 %
```






```
==============================
      TEST YOUR OWN IMAGE
==============================
Choose Files  Screenshot...ing.com.jpeg
Screenshot_7-10-2026_211028_www.bing.com.jpeg(image/jpeg) - 229900 bytes, last modified: 10/7/2026 - 100% done
Saving Screenshot_7-10-2026_211028_www.bing.com.jpeg to Screenshot_7-10-2026_211028_www.bing.com (1).jpeg

Image: Screenshot_7-10-2026_211028_www.bing.com (1).jpeg
Prediction: FRESH
Confidence: 88.67 %
```



Result

The CNN and MobileNetV2 models were successfully trained to classify fruits as **Fresh** or **Rotten**.The performance of both models was evaluated using test accuracy. The models were also compared using accuracy graphs, classification report, and confusion matrix.